

CO1 -- ASSESSMENT TOOL 3 SOLUTIONS

Name: Pavithra R

Register Number: 192421369

Course Code: ITA0404

Course Name: STATISTICS WITH R
PROGRAMMING

1. MATRIX OPERATIONS AND ANALYSIS

Objective & R Functions:

Create two 3x3 matrices with user-defined/sample values, perform addition (+), subtraction (-), matrix multiplication (%*%), transpose (t()), row/column sums (rowSums(), colSums()), and determinant (det()).

```
# Matrix Operations and Analysis

# 1. Create two 3x3 matrices
matrix_A <- matrix(c(2, 4, 3,
                     1, 5, 7,
                     6, 8, 9), nrow = 3, ncol = 3, byrow = TRUE)

matrix_B <- matrix(c(9, 8, 7,
                     6, 5, 4,
                     3, 2, 1), nrow = 3, ncol = 3, byrow = TRUE)

cat("=== Matrix A ===\n"); print(matrix_A)
cat("=== Matrix B ===\n"); print(matrix_B)

# 2. Arithmetic Operations
cat("=== Addition (A + B) ===\n"); print(matrix_A + matrix_B)
cat("=== Subtraction (A - B) ===\n"); print(matrix_A - matrix_B)
cat("=== Matrix Multiplication (A %*% B) ===\n"); print(matrix_A %*% matrix_B)

# 3. Transpose of each matrix
cat("=== Transpose of Matrix A ===\n"); print(t(matrix_A))
cat("=== Transpose of Matrix B ===\n"); print(t(matrix_B))

# 4. Analysis on Matrix A
cat("=== Matrix A Analysis ===\n")
cat("Row Sums:\n"); print(rowSums(matrix_A))
cat("Column Sums:\n"); print(colSums(matrix_A))
cat("Determinant of A:", det(matrix_A), "\n")
```

2. ARRAY OPERATIONS AND DATA ANALYSIS

Objective & R Functions:

Construct a 3D array (`array()`) representing student marks (3 Students x 3 Subjects x 2 Semesters). Calculate average marks per student using `apply()`, locate overall minimum and maximum (`min()`, `max()`), and slice sub-arrays by indexing.

```
# Array Operations and Data Analysis

# 1. Create a 3D Array [3 Students, 3 Subjects, 2 Semesters]
students <- c("Student_1", "Student_2", "Student_3")
subjects <- c("Math", "Physics", "R_Programming")
semesters <- c("Semester_1", "Semester_2")

# 18 total values (3x3x2)
marks_data <- c(85, 78, 92, 88, 74, 95, 80, 82, 89, # Semester 1
               90, 82, 88, 85, 79, 91, 84, 86, 93) # Semester 2

marks_array <- array(marks_data, dim = c(3, 3, 2),
                    dimnames = list(students, subjects, semesters))

cat("=== 3D Marks Array ===\n"); print(marks_array)

# 2. Calculate average marks for each student
# Margin 1 corresponds to rows (Students)
avg_marks <- apply(marks_array, 1, mean)
cat("=== Average Marks per Student ===\n"); print(avg_marks)

# 3. Highest and Lowest marks
cat("=== Summary Stats ===\n")
cat("Highest Mark in Array:", max(marks_array), "\n")
cat("Lowest Mark in Array :", min(marks_array), "\n")

# 4. Semester-wise and Subject-wise Indexing
cat("=== Semester 1 Marks ===\n"); print(marks_array[, , "Semester_1"])
cat("=== Semester 2 Marks ===\n"); print(marks_array[, , "Semester_2"])
cat("=== Subject-Wise Slicing (Math across all Sems) ===\n")
print(marks_array[, "Math", ])
```

3. R CALCULATIONS USING FUNCTIONS

Objective & R Functions:

Accept a 20-element vector, compute key statistical measures (`sum`, `mean`, `median`, `var`, `sd`, `max`, `min`, `range`), display formatted output, and define a custom comparison function.

```
# R Calculations Using Functions

# 1. Numeric Vector of 20 integers
data_vec <- c(45, 62, 78, 33, 89, 90, 54, 67, 72, 81,
              95, 48, 60, 77, 83, 88, 59, 64, 71, 92)

cat("=== Input Data Vector (20 Integers) ===
"); print(data_vec)

# 2 & 3. Compute and display summary statistics
display_stats <- function(vec) {
  cat("
=====
")
  cat("          STATISTICAL SUMMARY
")
  cat("=====
")
  cat(sprintf("%-20s : %d
", "Sum", sum(vec)))
  cat(sprintf("%-20s : %.2f
", "Mean", mean(vec)))
  cat(sprintf("%-20s : %.2f
", "Median", median(vec)))
  cat(sprintf("%-20s : %.2f
", "Variance", var(vec)))
  cat(sprintf("%-20s : %.2f
", "Std. Deviation", sd(vec)))
  cat(sprintf("%-20s : %d
", "Maximum", max(vec)))
  cat(sprintf("%-20s : %d
", "Minimum", min(vec)))
  cat(sprintf("%-20s : %d
", "Range", diff(range(vec))))
  cat("=====
")
}

display_stats(data_vec)

# 4. User-defined function to compare Mean and Median
compare_mean_median <- function(vec) {
  m <- mean(vec)
  med <- median(vec)
  cat("
=== Mean vs Median Evaluation ===
")
  cat(sprintf("Mean = %.2f | Median = %.2f
```

```
"", m, med))

  if (m > med) {
    cat("Status: Mean is GREATER than Median (Right-skewed distribution).
")
  } else if (m < med) {
    cat("Status: Mean is LESS than Median (Left-skewed distribution).
")
  } else {
    cat("Status: Mean is EQUAL to Median (Symmetric distribution).
")
  }
}

compare_mean_median(data_vec)
```

4. STUDENT MANAGEMENT SYSTEM USING S3 CLASS

Objective & R OOP Design:

Define an S3 class `Student` with attributes (ID, Name, Department, Marks). Implement custom generic/method functions (`calc\_average`, `assign\_grade`, `print.Student`).

```
# Student Management System Using S3 Class

# Constructor Function
Student <- function(id, name, dept, marks) {
  obj <- list(
    Student_ID = id,
    Name       = name,
    Department  = dept,
    Marks       = marks
  )
  class(obj) <- "Student"
  return(obj)
}

# S3 Generic for Average Calculation
calc_average <- function(obj) { UseMethod("calc_average") }
calc_average.Student <- function(obj) {
  return(mean(obj$Marks))
}

# S3 Generic for Grade Assignment
assign_grade <- function(obj) { UseMethod("assign_grade") }
assign_grade.Student <- function(obj) {
  avg <- calc_average(obj)
  if (avg >= 90) return("A+")
  else if (avg >= 80) return("A")
  else if (avg >= 70) return("B")
  else if (avg >= 60) return("C")
  else return("F")
}

# S3 Print Method
print.Student <- function(x, ...) {
  cat("=====")
  "
  cat("      STUDENT RECORD DETAILS
  "
  cat("=====")
  "
  cat("Student ID   :", x$Student_ID, "
  "
  cat("Name         :", x$Name, "
  "
  cat("Department   :", x$Department, "
  "
  cat("Marks        :", paste(x$Marks, collapse = ", "), "
  "
  cat("Average      :", sprintf("%.2f", calc_average(x)), "
```

```
)  
  cat("Grade      :", assign_grade(x), "  
)  
  cat("=====  
)  
}  
  
# Demonstration  
student_obj <- Student("S192421369", "Pavithra R", "Statistics", c(88, 92, 85, 90, 94))  
print(student_obj)
```

5. EMPLOYEE MANAGEMENT USING S4 AND REFERENCE CLASSES

Objective & R OOP Design:

Construct an S4 class `Employee` with validation check on `Salary > 0`. Build a Reference Class `Payroll` to mutate state and calculate annual salary, highlighting value vs reference semantics.

```
# Employee Management Using S4 and Reference Classes

# --- 1. S4 CLASS DEFINITION ---
setClass("Employee",
  slots = list(
    Emp_ID      = "character",
    Name        = "character",
    Department   = "character",
    Salary       = "numeric"
  ),
  validity = function(object) {
    if (object@Salary <= 0) {
      return("Salary must be a positive value!")
    }
    return(TRUE)
  }
)

# Instantiate S4 Object
emp_s4 <- new("Employee", Emp_ID = "E101", Name = "Pavithra R",
  Department = "Data Science", Salary = 65000)

cat("=== S4 Employee Object ===
")
print(emp_s4)

# --- 2. REFERENCE CLASS DEFINITION ---
Payroll <- setRefClass("Payroll",
  fields = list(
    Emp_ID      = "character",
    Name        = "character",
    Department   = "character",
    Salary       = "numeric"
  ),
  methods = list(
    updateSalary = function(new_salary) {
      if (new_salary > 0) {
        Salary <- new_salary
        cat("Salary updated to:", Salary, "
")
      } else {
        cat("Error: Invalid Salary!
")
      }
    },
    calculateAnnualSalary = function() {
      return(Salary * 12)
    },
  )
)
```



```

        display = function() {
            cat(sprintf("ID: %s | Name: %s | Monthly: %.2f | Annual: %.2f\n",
                        Emp_ID, Name, Salary, calculateAnnualSalary()))
        }
    )

# Instantiate Reference Class Object
emp_rc <- Payroll$new(Emp_ID = "E101", Name = "Pavithra R",
                     Department = "Data Science", Salary = 65000)

cat("
=== Reference Class Object ===
")
emp_rc$display()

# --- 3. DEMONSTRATING S4 (VALUE) VS RC (REFERENCE) SEMANTICS ---
cat("
=== DEMONSTRATING SEMANTIC DIFFERENCES ===
")

# S4: Pass-by-value (Copy on Modify)
emp_s4_copy <- emp_s4
emp_s4_copy@Salary <- 80000

cat("
[S4 Class Behavior - Copy-on-Modify]:
")
cat("Original S4 Salary:", emp_s4@Salary, "(Unchanged)
")
cat("Copied S4 Salary :", emp_s4_copy@Salary, "(Updated independently)
")

# Reference Class: Pass-by-reference (In-place Mutation)
emp_rc_ref <- emp_rc
emp_rc_ref$updateSalary(80000)

cat("
[Reference Class Behavior - Pass-by-Reference]:
")
cat("Original RC Salary :", emp_rc$Salary, "(Mutated in-place!)
")
cat("Referenced RC Salary :", emp_rc_ref$Salary, "(Points to same object)
")

```